

ES 116

Principles and Applications of Electrical Engineering

Multiplex Logic, Finite State Machines

EE Department
IIT Gandhinagar

Book References

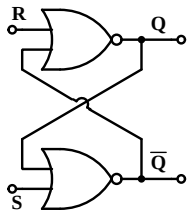
“Digital Systems” by Ronald J. Tocci, Prentice Hall, India
“Digital Fundamentals” by Thomas L. Floyd, Pearson Education.
“Digital Design” by M. Morris Mano, Pearson Education.

April 20, 2026

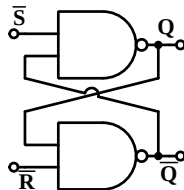
Recap ...

- In the previous lecture, we had discussed sequential logic, latches, flipflops, counters and Finite State Machines.
- We had also learnt about the evolution of microprocessors as general purpose digital circuits.
- In this lecture, we'll see how Finite State Machines are implemented and some alternative styles of realizing combinational logic.

Recap: RS latches



The cross-connected NOR (on the left) and the cross connected NAND (on the right) constitute RS latches which retain their set or reset state even after the activating input (R or S, for NOR latch, $\bar{R}$ or $\bar{S}$, for NAND latch) has gone away.



The active level is a '1' for the NOR RS latch and a '0' for the NAND RS latch.

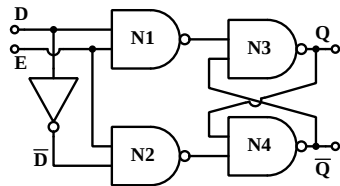
The output of these latches is unpredictable if both R and S go to '1' for the NOR latch, or both $\bar{R}$ and $\bar{S}$ go to '0' in case of NAND latch simultaneously and then return to their passive levels. **(Race Condition).**

Recap: Transparent D Latch

We can prevent the race condition by adding an inverter between R and S, as well as introducing an enable signal E which controls the transfer of the input to the latch.

When $E = 0$, outputs of NANDs N1 and N2 are forced to '1', so the inputs to the NAND latch formed by N3 and N4 are at their passive level ('1').

Thus for $E = 0$, the latch holds its old state.



When $E = 1$, $\bar{D}$ and D are applied to the latch formed by N3 and N4. This makes $Q = D$ and $\bar{Q} = \bar{D}$. This circuit is called a D (or Data) latch. It copies its input data D to the output while the Enable input is high. It retains this value when Enable goes low.

Thus when $E = 0$, Q holds the value of D which existed at the $1 \rightarrow 0$ transition of E, and when $E = 1$, $Q = D$.

Removing Transparency

- Consider a case when the D input to the latch is derived from its Q output. ($D = f(Q)$)
- When the latch is transparent, it forces Q to the value of D. ($Q = D$).
- This may not be consistent with $D = f(Q)$, and may lead to malfunction.

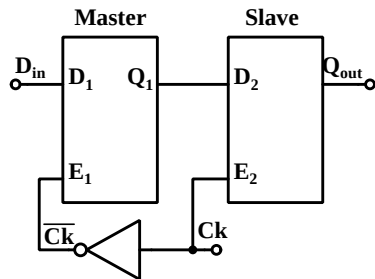
In most cases, we need a replica of the previous data as a snapshot taken at the instant of a clock transition, and **no transparency**.

- If the snapshot is taken at the positive transition of the clock ($0 \rightarrow 1$), the circuit is said to be positive edge triggered.
- If the snapshot is taken at the negative transition of the clock ($1 \rightarrow 0$), the circuit is said to be negative edge triggered.

Edge triggered memory elements (called flipflops) can be implemented using two transparent D latches enabled by complementary signals.

Edge triggered D flipflop

Consider two transparent D latches made from NAND gates as discussed earlier. These are latched when $E_1/E_2 = 0$ and transparent when $E_1/E_2 = 1$



When the clock is low, the first D latch is transparent. Output of the second latch is its previous value.

$$Q_1 = \overline{D}_{in}. \quad Q_2 = Q_{2prev}.$$

When the clock goes high, the first D latch captures the value at this instant, the second latch becomes transparent and outputs the value latched by the first D latch.

Thus, at any time, one of the two latches is non-transparent and the output does not follow the input. The output holds the value seen at D_{in} when the clock transitions from $0 \rightarrow 1$.

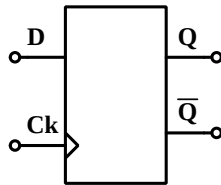
Flipflops as storage circuits: D flipflop

Flipflops are used in sequential circuits with a clock.

Their action occurs whenever the clock has a specified transition, say from '0' to '1'. This transition is known as the **active edge of the clock**.

D flipflop: This is a circuit with a 'D' (Data) input and a clock. At the active clock edge, it copies the value of D to its Q output and the complement appears at $\overline{Q}$.

The figure on the right shows the symbol used for a D flipflop.

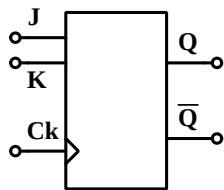


The little triangle at the clock input shows that the flipflop transfers D to Q on the rising edge of the clock. A little circle is placed within the triangular symbol if the active edge is the falling edge of the clock.

(Many circuit diagrams omit the indication for the active edge).

Recap: JK and T flipflops

JK flipflop: A JK flipflop has two inputs J and K, along with a clock. The J and K inputs act similarly to S and R inputs in the RS latch.



At the active edge of the clock,

If J = '0' and K = '0', the flipflop outputs remain unchanged.

If J = '1' and K = '0', Q goes to '1' and $\overline{Q}$ goes to '0'.

If J = '0' and K = '1', Q goes to '0' and $\overline{Q}$ goes to '1'.

If J = '1' and K = '1', it complements the values at Q and $\overline{Q}$.
(This and the presence of clock distinguish it from RS latch).

Toggle flipflop: In this, the J and K inputs are tied together and are used as the T input. When T = '0', the flipflop outputs Q and $\overline{Q}$ remain unchanged. When T = '1', it toggles *ie* complements the values at Q and $\overline{Q}$ at the active clock edge.

Making a JK flipflop from a D flipflop

Write the truth table for the desired next output of a JK flipflop and minimize it. (It can be given to the D input of a D flipflop to implement a JK flipflop).

If $J = '0'$ and $K = '0'$, Q and $\overline{Q}$ should remain unchanged.

If $J = '1'$ and $K = '0'$, $Q \rightarrow 1$ and $\overline{Q} \rightarrow 0$

If $J = '0'$ and $K = '1'$, $Q \rightarrow 0$ and $\overline{Q} \rightarrow 1$

If $J = '1'$ and $K = '1'$, values of Q and $\overline{Q}$ should be complemented.

Q	J	K	$D = Q_{next}$
0	0	0	?
0	0	1	?
0	1	0	?
0	1	1	?
1	0	0	?
1	0	1	?
1	1	0	?
1	1	1	?

Making a JK flipflop from a D flipflop

Q	J	K	$D = Q_{next}$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$D = Q_{next} = \overline{Q} \cdot J \cdot \overline{K} + \overline{Q} \cdot J \cdot K + Q \cdot \overline{J} \cdot \overline{K} + Q \cdot J \cdot \overline{K}$$

J, K ►

Q	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$D = J \cdot \overline{Q} + \overline{K} \cdot Q$$

Draw a logic diagram using NAND and NOR gates to implement this expression.

Shannon's Decomposition Theorem

Shannon's decomposition theorem (also known as Boole's expansion theorem) states that:

Given a Boolean function $F(x_1, x_2, \dots, x_n)$, we can express it as:

$$F(x_1, x_2, \dots, x_n) = x_i \cdot f_1 + \overline{x_i} \cdot f_2$$

where f_1 and f_2 are reduced expressions for F with x_i forced to 1 and 0 respectively. x_i is known as the pivot variable.

The theorem is equivalent to partitioning the truth table for F into two tables: One containing all the rows where the pivot variable $x_i = 1$, and the other containing all the rows where $x_i = 0$.

Then f_1 and f_2 are the functions corresponding to these two truth tables.

Multiplexer Based Logic

$$F(x_1, x_2, \dots, x_n) = x_i \cdot f1 + \overline{x_i} \cdot f2$$

where $f1$ and $f2$ are reduced expressions for F with x_i forced to 1 and 0 respectively.

- Thus, F can be implemented with a multiplexer controlled by x_i which selects $f1$ or $f2$ depending on x_i .
- $f1$ and $f2$ can themselves be decomposed into simpler expressions by the same technique.

Let us take a simple example – the xor function: $F = A\overline{B} + \overline{A}B$

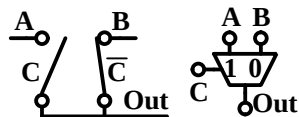
Using A as the pivot variable, $f1 = \overline{B}$, $f2 = B$.

Thus, a multiplexer choosing between $\overline{B}$ and B based on $A = 1$ or 0 will implement the xor function.

This can be used to implement the “sum” output of an adder.

Multiplexer Based Logic: Motivation

Why bother with mux based implementation of logic functions?



A multiplexer can be implemented quite easily in CMOS technology, using two switches controlled by complementary inputs.

Implementation of some hard to optimize logic functions (– such as the xor) is relatively simple using mux logic.

Switches can be implemented as single transistors.

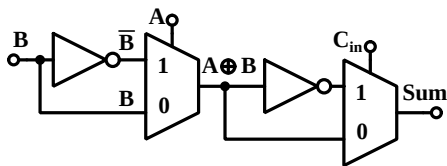
Multiplexer Based Logic ...

Let us implement the carry function for a full adder using multiplexers:

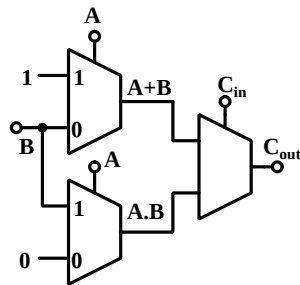
$C_{out} = AB + BC_{in} + C_{in}A$. Let us choose C_{in} as the pivot variable.

$f1 = AB + B + A = A + B$, $f2 = AB$.

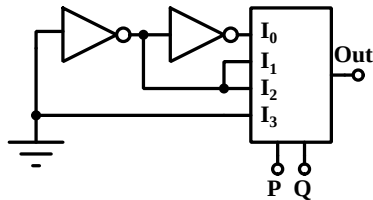
Thus choosing between the OR and AND of A and B using C_{in} will implement the output carry of a full adder. OR and AND functions can themselves be decomposed using Shannon's theorem.



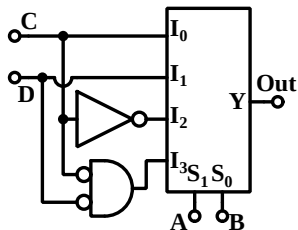
The circuits on the left and right show implementation of the sum and carry outputs of a full adder using multiplexers.



Practice Questions: Multiplexer Based Logic

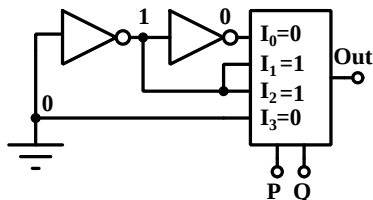


What function of P, Q is implemented by the mux circuit on the left?



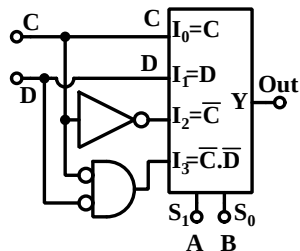
Which minterms of input variables A, B, C and D are included in the output SOP?

Practice Questions: Multiplexer Based Logic ...



P	Q	Out
0	0	0
0	1	1
1	0	1
1	1	0

$$\text{Out} = P \oplus Q$$



$$\text{Out} = \overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot \overline{B} \cdot D + A \cdot \overline{B} \cdot \overline{C} + A \cdot \overline{B} \cdot \overline{C} \cdot \overline{D}$$

Minterms must have all input variables in direct or complemented form.

$$= \overline{A} \overline{B} C(D + \overline{D}) + \overline{A} \overline{B} (C + \overline{C}) D + A \overline{B} \overline{C}(D + \overline{D}) + A \overline{B} \overline{C} \overline{D}$$

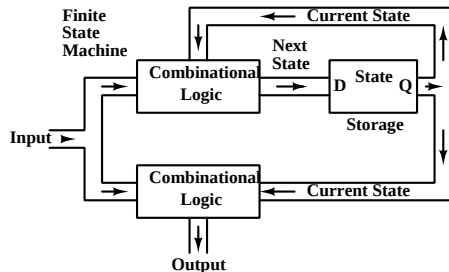
$$= m_3 + m_2 + m_7 + m_5 + m_9 + m_8 + m_{12}$$

Minterms 2, 3, 5, 7, 8, 9 and 12 are included in the output.

Finite State Machines

In a sequential circuit, the output sequence is a function of input sequence. The sequences between which we want to establish a functional relationship have to be of finite length to be implementable.

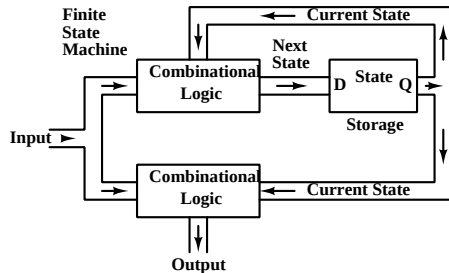
- A new set of input values are presented during each new clock period.
- It is not necessary to store all the past values of input variables.



A (possibly multi-bit) **state** is stored, which is a digital value with sufficient information about the past inputs so that the current element of the output sequence can be generated as a **combinational** function of the state and the current inputs.

Finite State Machines

- In each clock period, as fresh inputs arrive, a new value of **state** is generated which is a combinational function of the current value of state and current inputs.
- The state is thus a recursive function of previous inputs.

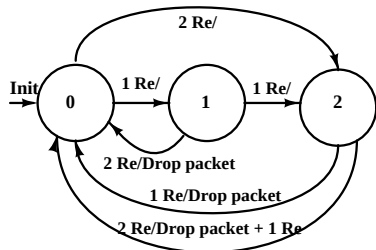


The **state** represents the “history” of input variables as relevant to production of the output sequence.

Systems which use this approach to generate sequential logic are called **Finite State Machines** or FSMs.

FSM example: Penny in the Slot Machine

Consider a Penny in the slot machine which accepts 1 Re and 2Re coins. It dispenses a packet worth 3 Rs. and returns coins if required.

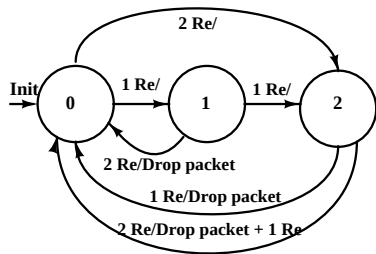


The state diagram on the left shows the evolution of 'states' in response to a sequence of inputs.

- At any time, one of two inputs may occur – insertion of a 1 Re coin or insertion of a 2 Re coin.
- In response, the machine may –
 - do nothing,
 - Drop the packet or
 - Drop the packet and a 1 Re coin.

The machine can be designed with 3 states and combinational logic to determine the action and the next state in response to specific inputs.

FSM example: Penny in the Slot Machine ...



At power on, the machine wakes up in state 0.

- At any time, The machine is in one of the three states shown.
- In any state, only two external inputs are possible: Arrival of a 1 Re coin or arrival of a 2 Re coin.

Curr st	Input	Output	Next St	Input	Output	Next St
0	1 Re	Nothing	1	2 Re	Nothing	2
1	1 Re	Nothing	2	2 Re	Drop packet	0
2	1 Re	Drop packet	0	2 Re	Drop packet + 1 Re	0

We can implement this with 2 flipflops to encode the state number and combinational logic to determine next state and output from current input and state.

Penny in the Slot Machine Implementation

State Table:

Q1	Q0	In1	In2	D1	D0
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	1	0
0	0	1	1	X	X
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	X	X

State Table:

Q1	Q0	In1	In2	D1	D0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	X	X
1	1	0	0	X	X
1	1	0	1	X	X
1	1	1	0	X	X
1	1	1	1	X	X

Simplifying using Karnaugh maps gives $D1 = In0 Q0 + In1 \overline{Q1} \overline{Q0} + Q1 \overline{In1} \overline{In0}$,
 $D0 = \overline{Q1} \overline{Q0} In0 + Q0 \overline{In1} \overline{In0}$

Penny in the Slot Machine Implementation ...

Action Table:

Q1	Q0	In1	In2	A1	A0
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	X	X
0	1	0	0	0	0
0	1	0	1	0	0
0	1	1	0	1	0
0	1	1	1	X	X

Action Table:

Q1	Q0	In1	In2	A1	A0
1	0	0	0	0	0
1	0	0	1	1	0
1	0	1	0	1	1
1	0	1	1	X	X
1	1	0	0	X	X
1	1	0	1	X	X
1	1	1	0	X	X
1	1	1	1	X	X

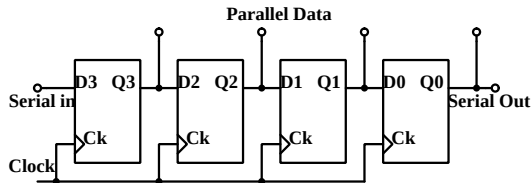
Simplifying using Karnaugh maps gives

(Drop Packet:) $A1 = (Q0 + Q1) \cdot In1 + Q1 \cdot In0$

(Drop 1 Re:) $A0 = Q1 \cdot In1$

Registers

- A group of D flipflops in parallel can be used to store a multi-bit value. This is called a **register**.
- We can also connect D flipflops in series as shown below. This arrangement is called a **Shift Register**.



At each active edge of the clock,
 $Q0 \rightarrow \text{Serial Out}$, $Q1 \rightarrow Q0$, $Q2 \rightarrow Q1$,
 $Q3 \rightarrow Q2$, and $\text{Serial In} \rightarrow Q3$.
 Thus data is shifted to the right by one location.

- This can be used for serial to parallel conversion by feeding data in serially and after all data has been fed, collecting it in parallel.
- Shift register are also used for parallel to serial conversion. Data is loaded in parallel to all the D flipflops and shifted out serially.
- These are also used for data delay, as a serial memory etc.

Number Systems

- We have already seen how we can represent natural numbers in a base-2 system (binary representation).

The bit pattern $b_{n-1} \dots b_i \dots b_1 b_0$ represents the number $N = \sum_{i=0}^{n-1} 2^i \cdot b_i$.

- Given a decimal number N , how do we determine b_i ?
 - Take N and divide by 2. The remainder is b_0 .
 - Repeat this process by dividing the quotient by 2 and retaining the remainder as $b_1, b_2 \dots$. Continue till the quotient becomes 1 or 0. This is then the most significant bit.
 - For somewhat larger numbers, it is more efficient to convert the number to base-16 or Hex format, by using the above procedure but dividing by 16 every time.
 - The resulting Hex number can be converted to binary easily by replacing each hex digit by its binary equivalent.

An n bit binary number can represent any natural number in the range $0 \leq N \leq 2^n - 1$

Representation of Natural numbers

Let us illustrate the conversion procedure by a few examples.

Given $N = 55$.

Divide successively by 2.

$$55 = 27 * 2 + 1, \text{ so } b_0 = 1$$

$$27 = 13 * 2 + 1, \text{ so } b_1 = 1$$

$$13 = 6 * 2 + 1, \text{ so } b_2 = 1$$

$$6 = 3 * 2 + 0, \text{ so } b_3 = 0$$

$$3 = 1 * 2 + 1, \text{ so } b_4 = 1$$

$$\text{Quotient} = 1, \text{ so } b_5 = 1$$

Thus decimal $55 = \text{binary } 111011$

Given $N = 1000$. Divide by 16.

$$1000 = 62 * 16 + 8,$$

so least significant hex digit is 8.

$$62 = 3 * 16 + 14,$$

so the next digit is 14 or E.

$3 < 16$, so the most significant digit is 3. Thus $1000 = 3E8$ in Hex.

Expanding each hex digit to binary, this can be written as:

$$0011 \mid 1110 \mid 1000.$$

Binary addition

- Binary numbers can be added just like decimal numbers, with a carry being transferred to a more significant bit position whenever the sum exceeds 1.
- We can represent positive numbers between 0 and $2^n - 1$ using n bits.
- What happens if the sum exceeds the largest representable number?

Consider an example using 4 bit numbers. Using 4 bits we can represent numbers between 0 and 15

Now consider the addition of 11 to 7 in binary arithmetic using 4 bits.

$1011 + 0111 = 10010$. The overflowing 5th bit can be used to signal that overflow has occurred and the result is not valid.

However, this provides us with a clue to a method for representing negative numbers.

Representing Negative numbers

- In arithmetic, we define a negative number by the property: $x + (-x) = 0$.
- To represent negative numbers, we take the magnitude of the number and then choose a number which when added to it will give zeros in the defined bit width, (albeit with an overflow).
- For example, the decimal number 5 is 0101 using 4 bit representation. When we add decimal 11 to it we get $0101 + 1011 = 10000$. ($5+11=16$). If we ignore the overflow (fifth bit), the result is zero.
- Thus, if $x = 0101$, $(-x) = 1011$. Now, $x + (-x) = 0$, if we ignore the overflow.
- Obviously, we cannot use 1011 to represent 11 as well as -5!
So we adopt a convention where numbers with 0 in the most significant position are positive, while numbers with 1 in the most significant position represent negative numbers.

Representing Negative numbers

- We use the convention where numbers with 0 in the most significant position are positive, while numbers with 1 in the most significant position represent negative numbers.
- With this convention, signed 4 bit numbers can be represented in the range -8 to + 7, inclusive of Zero.
- How do we find the number which when added to x will give zero (with overflow)?
- Notice that when we add a number to its complement (changing all 0's to 1 and 1's to zero), the sum will have 1 in all positions.
Now if add 1 to this, we'll get all zeros with an overflow.
- So to get the negative of a number, we take its complement (also called 1's complement) and add 1 to it. (This is called 2's complement).

Representing Fractional Numbers

Recall the problem which we had solved during the tutorial ...

Let us take a similar example where we want to represent 17.65 with 5 bits for the integral part and 8 bits for the fractional part.

We first represent 17 (5 bits) using successive division by 2.

Dividend	Quotient	Remainder
17	8	1
8	4	0
4	2	0

Dividend	Quotient	Remainder
2	1	0
1	0	1

So 17 base 10 = 10001 base 2

Representing the Fractional Part

There is an implied binary point after 5 bits, and bits to the right of it have place values which are negative powers of 2. Let us convert the fractional part (0.65) to the binary fractional part.

Multiplicand	Fraction	Integer
0.65	0.3	1
0.3	0.6	0
0.6	0.2	1
0.2	0.4	0

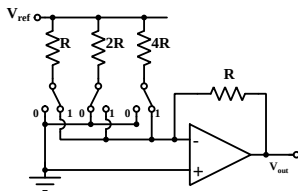
Multiplicand	Fraction	Integer
0.4	0.8	0
0.8	0.6	1
0.6	0.2	1
0.2	0.4	0

So 17.65 will be represented as 10001.10100110.

How will you represent -17.65?

Digital to Analog Conversion

Consider the production of an analog quantity – say voltage, proportional to a given digital numbers. This is Digital to Analog conversion or DAC.



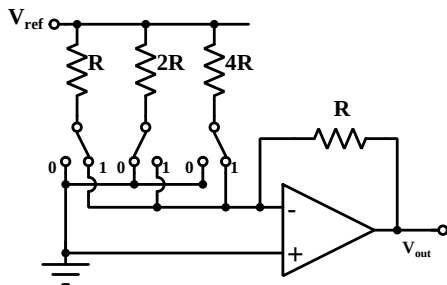
A digital number: $b_{n-1} \dots b_i \dots b_1 b_0$
represents $\sum_{i=0}^{n-1} 2^i b_i$.

If we use resistors with resistance values proportional to 2^i and use these to draw current from a reference voltage V_{ref} (with the other end of the resistors at ground potential), the currents will be in binary ratio.

We put a two way switch at the end other than V_{ref} which connects either to ground or to the virtual ground of an opamp.

Since these two choices are at the same potential, the current remains unchanged if we switch it between either of these.

Digital to Analog Conversion



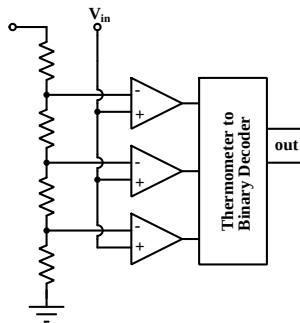
Each two way switch is controlled by a bit of the digital number which we want to convert. Since the two way switches direct the current either to ground or to the virtual ground of an opamp, the current through the resistors remains unchanged if we switch it between either of these.

The current going into the inverting terminal is then proportional to the digital number. The output voltage is $-R$ times this current and thus proportional to the digital number.

This method of D to A conversion is intuitive, but getting accurate values of resistors over a wide range is a difficult task.

Analog to Digital Conversion

A flash ADC is a fast converter which uses a separate comparator for each digital value possible. A resistor divider creates equally spaced $N-1$ reference levels.



- A bank of comparators compares the input to each of the reference levels.
- All comparators with reference level below the input value go to '1' while all those above it go to '0'.
This is known as **Thermometer coding**.
- A decoder is then used to transform the thermometer code to binary.

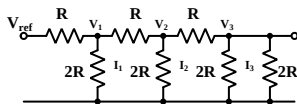
Flash converters are used for A to D conversion with a small number of bits, since its complexity grows exponentially with each additional bit.

Analog to Digital Conversion

- Many other types of A to D converters are available, providing a compromise between speed, power and complexity.
- A successive approximation ADC uses a DAC whose Digital input is provided by a register. The ADC sequentially adjusts the bits of this register and compares the output of the DAC with the input voltage, till they match. This type of ADC is widely used in medium speed applications.
- A dual slope ADC provides low complexity at low conversion rates. It converts its input voltage to a time interval, which is measured using a counter. This kind of ADC is used by most DMMs.

From the user's view point, a “start conversion” signal is sent to the ADC (of whatever type) and when the conversion is complete, it sends an “end of conversion” signal. The output of the ADC can be read as a digital number once the conversion is complete.

DAC with R-2R ladder network



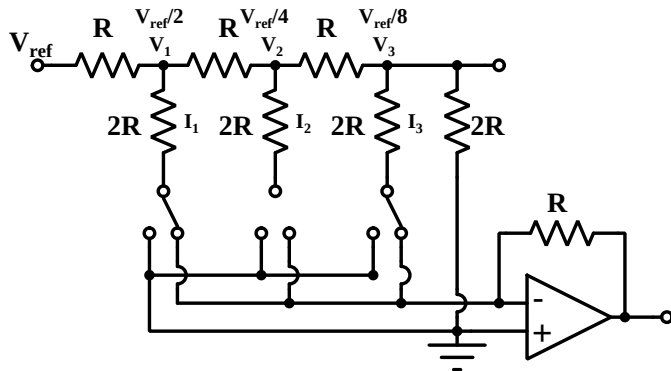
Consider a network with resistors of value R in series and those of value $2R$ in parallel in a ladder network as shown on the left.

Notice the extra $2R$ resistor at the end of the chain.

- At the end, we have 2 resistors to ground, each of value $2R$. This is equivalent to a single resistor of value R .
- This equivalent resistance of R and the series resistance R from V_2 form a potential divider such that $V_3 = V_2/2$.
- The series resistor from V_2 and the equivalent resistor R to ground provide a total resistance of $2R$ to ground from V_2 .
- Extending this argument, $V_2 = V_1/2$ and $V_1 = V_{ref}/2$
- Correspondingly, the current to ground through the $2R$ resistors is in binary ratio.
- by inserting a two way switch at the ground end and switching between ground and virtual ground, we can generate a voltage proportional to the digital value.

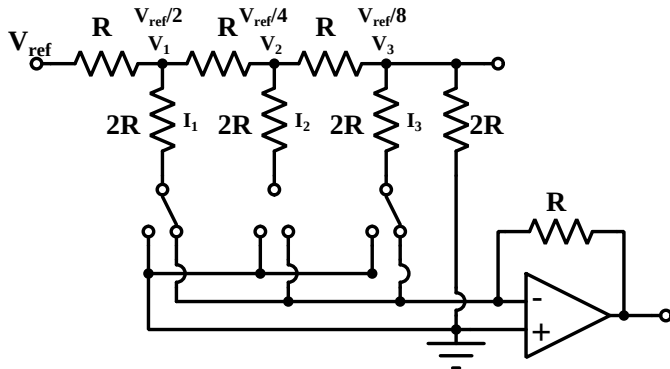
DAC with R-2R ladder network

The Circuit below shows a D to A converter using the R-2R ladder network.



- Bits of the digital value choose whether the current through the $2R$ resistors will go to ground or to virtual ground.
- Total current going into the virtual ground is converted to a voltage using a feedback resistor.

DAC with R-2R ladder network . . .



- Implementing the DAC is easier this way, because it requires only two resistance values. In fact, two identical resistors are put in series to get the $2R$ value.
- Accurate matching of resistors is much easier here compared to the binary weighted resistor case.

ADC with Successive Approximation

- This is a widely used architecture for ADCs.
- It uses a DAC and a comparator.
- At the first step, the most significant bit for the DAC is set.
- The comparator indicates if the unknown voltage is above or below the DAC output. If the unknown value is below the DAC value, the bit which was last set is cleared.
- Now the next significant bit for the DAC input is set. The unknown voltage is compared to the DAC output again.
- This process is continued till the least significant bit has been determined.

Successive Approximation Register (SAR) ADC example

Consider an example case where $V_{ref} = 1.0 \text{ V}$ and the register as well as DAC are 8 bits wide. Assume the input to be 0.65 V . Least count of the DAC is $1/2^8 = 1/256 \text{ V}$. To begin with, we set the most significant bit (b7) which has a place value of 128.

SA Reg.	DAC out (V)	Compar.	Action
10000000	$128/256 = 0.5$	1	Leave b7 set, set b6
11000000	$(128+64)/256 = 0.75$	0	reset b6, set b5
10100000	$(128+32)/256 = 0.625$	1	Leave b5 set, set b4
10110000	$(128+32+16)/256 = 0.6875$	0	reset b4, set b3
10101000	$(128+32+8)/256 = 0.65625$	0	reset b3, set b2
10100100	$(128+32+4)/256 = 0.640625$	1	leave b2 set, set b1
10100110	$(128+32+4+2)/256 = 0.6484375$	1	leave b1 set, set b0
10100111	$(128+32+4+2+1)/256 = 0.65234375$	0	reset b0, done

ADC output = 10100110 = 166, equivalent to 0.6484375 V .

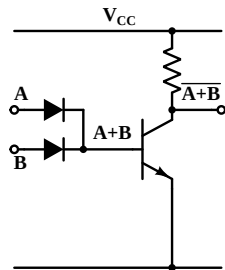
Quantization error = $0.65 - 0.6484375 = 1.5625 \text{ mV}$.

Logic families: DTL

Earliest implementation of Digital logic used relays connected in series/parallel to implement logic.

With the advent of semiconductor devices, diodes and transistors were used to implement digital logic. The circuit on the right is a DTL NOR gate.

A HIGH voltage (close to V_{CC}) represents a logic '1', while a LOW voltage (close to ground) represents a logic '0'.



When either A or B (or both) are HIGH, base current flows, saturating the bipolar transistor. This pulls the output LOW.

The transistor is OFF when both A and B are LOW as there is no base current. The output is then connected to V_{CC} through the collector resistor and is HIGH.

Logic families: TTL and CMOS

- Diode-Transistor Logic was eventually replaced by Transistor-Transistor Logic or TTL. This was the dominant digital technology for implementation of digital logic for a long time.
- TTL replaces the diodes at the input by a bipolar transistor with multiple emitters. In this course, we shall not get into the internal circuitry of TTL.
- Modern digital circuits use MOS transistors. By using complementary MOS transistor types (n channel and p channel), high speed can be obtained at reasonably low power consumption. This is known as CMOS logic.
- With advances in integrated circuit technology, multiple logic functions (and indeed entire micro-computers) can be placed on a single integrated circuit.